In-Memory File System: Requirements Checklist

UTEP CS 1301, Project 2 (assignment page: Intro: In-Memory File System Overview). Compiled from the assignment page and Mr. Nava's two email replies. It restates the spec and records your decisions and his answers (section 10). The one code block in it, the Command interface, is printed exactly as the assignment gives it.

Course rules that decide everything else

- **Submitting fully or partially AI-generated code is not allowed for this project.**
- Missing a required check-in or the demo meeting results in a **zero for the entire project**.
- Late submissions will not be accepted. Accommodations are evaluated case by case.

1. Key dates

Event	When
Project co-working session (optional)	Wednesday, October 28, 1 to 4 PM, Room 2.0902A inside Research Lab 2.0902
Individual check-in (mandatory, 7 minutes)	10/29 to 11/4. Mr. Nava will open the time slots by Monday, October 26 at the latest.
Final code submission	**Sunday, November 8, 11:59 PM**
Individual demo meeting (mandatory, 20 minutes)	To be announced

2. What Mr. Nava's email allows

You asked whether you could implement everything from scratch instead of building on the provided helper code. He replied: *"Yes, that is perfectly fine. As long as it covers what's in the Required Functionality section, you can feel free to implement it from scratch."* His link points to the start of the Required functionality table.

What that means in practice (my reading of the page):

- You have decided to implement it from scratch and keep the assignment's class names (section 10).
- Everything the starter provided becomes yours to write: the entry point and terminal command loop; command parsing, help messages and argument-count checks; the file and folder fields and constructors; folder navigation, name validation and path helpers; and the operations the commands call.
- The assignment says the supplied switch-based design is "a starting point, not the required final design." Whichever way you start, the final submission must use polymorphic command dispatch (section 6).
- The page says commands already implemented in the starter must continue to work. Writing from scratch, that means all 15 commands in section 3 must work in your version.
- On the page, the headings Disk storage requirements, Names, paths, and required validation, and Command classes, composition, and polymorphism sit under the Required functionality heading, so sections 4 to 6 below are all treated as required.

3. Required commands (15)

Command	Required behavior	Done
`mkdir NAME`	Create a direct subfolder in the current folder.	[ ]
`touch NAME SIZE_KB`	Create a simulated file with the specified size and allocate its disk space.	[ ]
`ls`	List the current folder's direct files and subfolders.	[ ]
`ls -R`	List the current folder and all descendant folders recursively, with a full path heading for each folder.	[ ]
`cd NAME`	Enter a direct subfolder.	[ ]
`cd ..`	Move to the parent folder. At the root, remain at the root.	[ ]
`cd /`	Return to the root folder.	[ ]
`pwd`	Display the current folder's full path.	[ ]
`size`	Display the total size of files in the current folder and all its descendants.	[ ]
`size NAME`	Display the size of a direct file, or the recursive total size of a direct subfolder.	[ ]
`find NAME`	Search the current folder and all descendants for files whose names match exactly. Display each matching full path and the number of matches.	[ ]
`rm NAME`	Delete a direct file or folder. Deleting a folder must also delete its descendants and reclaim their disk space.	[ ]
`disk`	Display the simulated disk and its used and available space.	[ ]
`help`	Display command usage information.	[ ]
`exit`	End the program.	[ ]

4. Disk storage requirements

[ ] Represent disk storage with an `int[][]`.

[ ] The page says the initial disk is **5 rows by 10 columns**. Mr. Nava's reply lets you define the size yourself as long as it is a 2D matrix, or ask the user to enter it. Each cell represents **1 KB**.

[ ] A cell value of `0` means the cell is available. A positive value identifies the file occupying that cell.

[ ] Allocate available cells in row order. A file's blocks do not need to be contiguous.

[ ] Before allocating a file, make sure enough space is available. A failed allocation must leave the disk and the file map unchanged.

[ ] A zero-size file is valid and occupies no disk cells.

[ ] Every successfully created file receives a unique positive ID. Deleted IDs are not reused.

[ ] Deleting a file releases all cells containing its ID.

[ ] Deleting a folder releases the storage belonging to every file in its subtree.

[ ] Display free cells as `.` and occupied cells using their file IDs. (Output layout and wording are yours: Mr. Nava will not auto-grade output, only functionality.)

5. Names, paths, and required validation

The page says to "keep the provided validation and handle these cases." Writing from scratch, you write that validation yourself.

- [ ] Names are case-sensitive.
- [ ] Names cannot contain spaces, `/`, or `\`.
- [ ] `.` and `..` are reserved names.
- [ ] A file and a folder cannot share a name within the same folder.
- [ ] Different folders may contain files with the same name.
- [ ] File sizes must be nonnegative integers.
- [ ] Missing files or folders must produce a clear message.
- [ ] Invalid input must not create partial files, consume disk space, or delete unrelated entries.
- [ ] Root folder paths: the root's path is `/`, so joining it with `docs` must produce `/docs`, not `//docs`. (The starter's `childPath()` helper did this; yours must too.)

6. Command classes, composition, and polymorphism

- [ ] One class per command: `CreateFolderCommand`, `CreateFileCommand`, `ListCommand`, `ChangeFolderCommand`, `PrintPathCommand`, `SizeCommand`, `FindCommand`, `RemoveCommand`, `DiskCommand`, `HelpCommand`, and `ExitCommand` (11 classes).
- [ ] Every command class implements the shared `Command` interface (below).
- [ ] The argument count and the argument array exclude the command name.
- [ ] Use composition to give each command the objects it needs, such as `FileSystem` or `Disk`. Pass these dependencies through constructors.
- [ ] Each command provides its usage text, description, argument-count validation, and execution behavior.
- [ ] Keep fields private and expose appropriate methods rather than making all program state public.
- [ ] Conditionals inside an individual command (for example, telling `ls` from `ls -R`) are permitted.
- [ ] `FileSystem` registers the command objects in a `Map<String, Command>`, looks up the requested command, and invokes it through the interface.
- [ ] Help is generated from the registered command objects.
- [ ] **The final submission must use polymorphic command dispatch.** A command-selection `switch` or an equivalent `if/else` chain does not satisfy this requirement.

The interface, exactly as the assignment prints it:

```
public interface Command {
    String usage();
    String description();
    boolean accepts(int argumentCount);
    void execute(String[] arguments);
}
```

7. Scope: what you do not need to build

These commands are simplified. Their behavior is defined by this project, not by Linux or another operating system. For example, `touch NAME SIZE_KB` creates a simulated file with a size; it does not need to reproduce Linux `touch`.

You do not need to implement:

- Full-path navigation such as `cd /folder1/folder2`.

- Paths as arguments to other operations, such as `rm notes/todo.txt`.
- Names containing spaces, quoted names, or escaped characters.
- File contents, file editing, or timestamps.
- Wildcards, partial matching, or regular expressions for `find`.
- Additional command flags beyond those specified.
- Alphabetically sorted listings.
- Operating-system permissions or other platform-specific filename rules.
- Persistent storage, file input/output, databases, GUIs, or external libraries.

Method requirements stated on the page: required recursive operations must use recursion; use nested loops for disk processing; use maps for the file and folder collections. The simulation is in-memory only: it must not create, modify, or delete real files on your computer, and everything disappears when the program exits.

8. Meetings, grading, and submission

Required check-in (7 minutes, individual, with the instructor, TA, or Tech Fellow; window 10/29 to 11/4)

- Be prepared to run your current program, explain your implementation, and identify the remaining work.
- Show working functionality rather than only describing what you intend to build.

Required demo and personalized twist (20 minutes, individual, date to be announced)

- **First 3 minutes:** explain your code, including class responsibilities, important helper methods, and your recursive operations.
- **Remaining 17 minutes:** implement a personalized change or new feature assigned during the meeting. The twist is selected individually for each student.
- You are expected to understand your submitted code well enough to locate the relevant logic, make changes, and explain your reasoning.
- **Failure to complete the twist may result in zero points for both the Code and Demo categories**, in addition to the points earned or lost for the Twist category.

Component	Points
Checkpoint	15
Code	50
Demo	15
Personalized twist	20
Total	**100**

Final submission: all Java source files needed to compile and run the project.

9. Proposed timeline (from the page; a reference, "not written in stone")

Date	Class topics	Functionality you may want to implement by then	Skills practiced
Thu, Oct 22	Recursion: previously covered. Maps: Oct 20-22.	Create folders. List direct contents. List nested folders recursively. Demonstrate navigation using the provided implementation.	Map insertion and iteration; recursion; using provided classes; method decomposition
Sun, Oct 25	Nested loops: Oct 20-22. 2D arrays: Oct 20-22.	Allocate disk blocks. Reject allocations when space is insufficient. Display the disk and storage totals. Design supporting helpers.	2D arrays; nested loops; counters and conditionals; decomposition
Sun, Nov 1	Classes: Oct 27-29.	Create files and allocate their storage. Include files in listings. Implement size queries and recursive search.	Classes and objects; maps; recursion; combining components
Thu, Nov 5	Application and review of covered topics.	Delete files and folders. Reclaim storage recursively. Demonstrate that reclaimed space can be reused. Replace switch-based command selection with individual classes implementing Command, registered in a command map and executed through polymorphism.	Map removal; recursion; 2D arrays; nested loops; object collaboration
Sun, Nov 8	Consolidation.	Submit the complete project. Verify required behavior and edge cases.	All project learning goals

My reading, not stated on the page: the check-in window (10/29 to 11/4) falls between the Oct 25 and Nov 1 rows, so having folders, listing, navigation, and disk allocation running by the check-in is a sensible target.

10. Your decisions, Mr. Nava's answers, and what is still open

Decisions you have made (yours, not Mr. Nava's):

- Implement the project from scratch, without the starter code, as Mr. Nava's email allows.
- Keep the class names from the assignment's examples: Main, FileSystem, File, Folder, Disk, the Command interface, and the 11 command classes.
- All 15 commands are yours to implement, so which commands the starter already implemented no longer matters.
- A failed touch and file IDs: you see this as clear from the spec (only a successfully created file gets an ID, and a failed allocation changes nothing). Not asked.
- The 15 Checkpoint points are earned through each step, such as the check-in meeting with Mr. Nava.

Mr. Nava's answers (second email reply, Thursday, October 22, 3:54 PM):

- **Output:** he will not do automated grading, so you can format the output however you prefer. Only the functionality is graded.
- **Disk size:** you can define the disk size yourself, as long as it is a 2D matrix. You can also ask the user to input the disk size. Up to you.
- **Check-in time slots:** not open yet. He will open them by Monday, October 26 at the latest.

Still open: nothing from the questions you sent. Watch your email on Monday for the check-in time slots, and for the demo meeting date, which is still to be announced.